UNIVERSIDAD AUTÓNOMA "GABRIEL RENÉ MORENO"
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES



Proyecto:

"Ciudad Interactiva con OpenGL"

Estudiante: Rosales Colque Angeles

Registro: 223044172

Materia: Programación Gráfica

Grupo: SA

Docente: MSc. Ing. Kenji Kawaida Villegas

GESTIÓN 2-2026

ÍNDICE

1. INTRODUCCIÓN 3
2. OBJETIVOS 3
3. HERRAMIENTAS UTILIZADAS 3
4. DISEÑO Y ORGANIZACIÓN DEL PROYECTO 4
5. FUNCIONES IMPLEMENTADAS 4
6. PROCESO DE APLICACIÓN DE LAS MEJORAS 5
7. CONTROLES E INSTRUCCIONES DE EJECUCIÓN 5
8. PRUEBAS Y LIMITACIONES 6
9. RECURSOS 6
10. CONCLUSIÓN 7
11. BIBLIOGRAFÍA 8

1. INTRODUCCIÓN

El proyecto consiste en la ampliación de una ciudad interactiva desarrollada en Java y OpenGL, a partir del código proporcionado por el docente. La aplicación permite conducir un automóvil por un escenario con calles, edificios, parques, iluminación y zonas de entrega.

Las mejoras incluyen una ciudad más extensa, tráfico autónomo, semáforos, minimapa, sonidos y una interfaz con menús e indicadores. Se conservó la organización del proyecto original y se incorporaron clases auxiliares para separar las nuevas responsabilidades.

2. OBJETIVOS

Ampliar y mejorar la ciudad interactiva aplicando conceptos de programación gráfica, como transformaciones, cámaras, iluminación y detección de colisiones, para ofrecer una experiencia de conducción más completa y cumplir los requisitos del enunciado.

3. HERRAMIENTAS UTILIZADAS

Java se utilizó para programar la conducción, el tráfico, las colisiones y las reglas de las entregas. El proyecto mantiene compatibilidad con Java 17 y utiliza herencia para ampliar las clases originales.

Maven permitió administrar las dependencias y ejecutar los procesos de compilación, pruebas y empaquetado mediante el archivo pom.xml.

LWJGL y GLFW proporcionaron el acceso a OpenGL, la creación de la ventana y la lectura de las entradas del teclado y del ratón.

OpenGL y GLSL se utilizaron para dibujar los objetos tridimensionales y calcular su iluminación. Los shaders procesan las posiciones, normales y colores de las superficies para representar la luz ambiental, las farolas y los faros del automóvil.

Java2D permitió generar los textos, botones, paneles y la flecha de navegación. Estos elementos se convierten en texturas que OpenGL muestra sobre la escena.

Java Sound se empleó para generar los sonidos del motor, frenado, choques y entregas, sin utilizar archivos de audio descargados.

JUnit permitió realizar pruebas automatizadas para comprobar la conducción, las colisiones, el tráfico, las entregas y el funcionamiento de la interfaz.

Visual Studio Code se utilizó como editor para organizar los archivos, modificar el código y revisar los mensajes de compilación.

4. DISEÑO Y ORGANIZACIÓN DEL PROYECTO

La ciudad utiliza una matriz de 11 × 11 celdas, conservando el tamaño de 10 unidades por celda. Los valores de la matriz identifican calles, edificios y parques.

El escenario contiene 21 manzanas con construcciones y 4 parques. Se incorporaron casas, comercios, departamentos y torres con diferentes dimensiones y colores. Las calles conectan las tres zonas de entrega.

Se conservó la organización principal del docente:

- **clase1:** ventana, geometría, ciudad y dibujo básico.
- **clase2:** conducción, cámara y colisiones del vehículo.
- **clase3:** iluminación diurna y nocturna, farolas y faros.
- **clase4:** entregas, tráfico, semáforos, minimapa e interfaz.

Las clases auxiliares separan el tráfico, los cruces, los modelos, las colisiones, los paneles y el sonido. La geometría se construye mediante código, utilizando cajas, prismas y cilindros.

5. FUNCIONES IMPLEMENTADAS

Las funciones principales son:

- **Conducción:** avance, reversa, giro y frenado mediante teclado o botón en pantalla.
- **Colisiones:** detección de edificios, parques, vehículos y límites del mapa. Las aceras permiten circular.
- **Cámaras:** vista general y seguimiento cercano con altura y orientación ajustables.
- **Iluminación:** cambio entre día y noche, 25 farolas y dos faros delanteros que acompañan al auto.
- **Decoración:** árboles, bancos, ventanas, pasos peatonales y señalización.
- **Semáforos:** secuencia de colores controlada por tiempo.
- **Tráfico autónomo:** tres vehículos que recorren las calles, respetan los semáforos y comprueban que tengan espacio para pasar.
- **Minimapa:** muestra la ciudad, el vehículo y el destino activo con el norte hacia arriba.
- **Entregas:** tres destinos señalados mediante un aro animado y una flecha. Para entregar se debe llegar y reducir la velocidad.
- **Interfaz:** pantalla de inicio, pausa, indicadores, ayuda y resumen final con tiempo, entregas y choques.
- **Sonido:** motor, frenado, contacto y confirmación de entregas, con opción de silencio.

La versión actual también incluye una chata articulada que acompaña el movimiento del vehículo.

6. PROCESO DE APLICACIÓN DE LAS MEJORAS

Primero se revisó el proyecto original y el enunciado. Se identificaron las funciones existentes y los componentes que debían ampliarse, conservando la estructura de trabajo del docente.

Después se amplió la ciudad. Se modificó la matriz del mapa y se adaptaron el suelo, los límites, las cámaras y la escala del minimapa para representar el escenario completo.

Luego se mejoró la presentación visual. Se añadieron distintos modelos de construcciones, decoración y un auto deportivo. Se ajustaron las posiciones de las luces para que coincidieran con los elementos dibujados.

Se incorporaron las funciones de circulación. El tráfico se organizó en vehículos con estado y ruta propios. Los semáforos y los autos consultan el mismo reloj, de modo que las luces visibles coinciden con las reglas de paso.

Finalmente se mejoró la interacción. Se añadieron menús, sonido, avisos, navegación y estadísticas. Las modificaciones se comprobaron por partes mediante pruebas automatizadas y ejecuciones gráficas.

El movimiento utiliza deltaTime, que representa el tiempo transcurrido entre cuadros, para calcular los desplazamientos según el tiempo.

7. CONTROLES E INSTRUCCIONES DE EJECUCIÓN

Control	Acción
Enter	Iniciar o continuar
W / S o flechas arriba / abajo	Avanzar y retroceder
A / D o flechas izquierda / derecha	Girar
Espacio o botón FRENO	Frenar
C	Cambiar cámara
Ratón derecho y arrastre	Ajustar la vista cercana
Re Pág / Av Pág	Subir o bajar la cámara
Inicio	Restablecer la vista cercana
N	Cambiar día y noche
F	Encender o apagar los faros
M	Mostrar u ocultar el minimapa
H	Mostrar u ocultar la ayuda
P	Pausar
V	Activar o silenciar el audio
R	Reiniciar la partida
Esc	Salir

Para ejecutar se necesita un JDK 17 o superior, Maven y soporte gráfico para OpenGL 3.3.

Desde la carpeta que contiene pom.xml, ejecutar:
mvn compile exec:exec -Pclase4

La primera ejecución requiere conexión a Internet para descargar las dependencias. Para ejecutar las pruebas: mvn test

La entrega debe incluir las fuentes, pom.xml, documentación y archivos necesarios. Deben excluirse los archivos temporales, la carpeta target y las dependencias descargadas.

8. PRUEBAS Y LIMITACIONES

Se ejecutaron 43 pruebas automatizadas sobre conducción, colisiones, tráfico, entregas, reinicio, menús, navegación y estadísticas. También se realizaron verificaciones gráficas de las cuatro etapas, los controles y el redimensionamiento de la ventana.

El proyecto presenta las siguientes limitaciones:

- Los vehículos autónomos siguen rutas fijas y esperan si están bloqueados.
- Las colisiones utilizan formas geométricas aproximadas.
- La iluminación no calcula sombras ni oclusión.
- La flecha apunta directamente al destino; no calcula una ruta por las calles.
- La subida a las aceras utiliza una elevación sencilla, sin suspensión.
- El audio es sintetizado y depende de la disponibilidad del dispositivo de sonido.

9. RECURSOS

El punto de partida fue el proyecto proporcionado por el docente y el enunciado de la actividad.

Se utilizaron las bibliotecas LWJGL, GLFW y JUnit, junto con las herramientas incluidas en Java. Los modelos, la interfaz y los sonidos se generan mediante código; no se incorporaron imágenes, modelos tridimensionales ni pistas de audio descargadas.

Se contó con asistencia de inteligencia artificial para apoyar la implementación, revisión y explicación de las mejoras.

10. CONCLUSIÓN

El desarrollo de la ciudad interactiva permitió aplicar los contenidos de la materia en un proyecto que combina escenarios tridimensionales, iluminación, movimiento y controles. A partir del código proporcionado por el docente, se amplió la ciudad y se incorporaron mejoras que hicieron más completa la experiencia de conducción, manteniendo la organización del proyecto original.

Conocer OpenGL, LWJGL y GLSL fue una experiencia nueva en mi formación, ya que hasta este momento no había trabajado con estas herramientas en otras materias cursadas. Su uso me permitió comprender cómo el código puede construir objetos, transformar sus posiciones y calcular la iluminación de una escena. También pude relacionar conocimientos de programación y matemáticas con resultados visuales e interactivos.

Este aprendizaje fue importante para mi carrera porque amplió mi visión sobre las aplicaciones de la programación en áreas como los videojuegos, los simuladores y la visualización de información. Además, el proceso de probar, detectar errores y ajustar las mejoras fortaleció mi capacidad para analizar problemas y organizar soluciones. El proyecto representa una primera base para continuar aprendiendo sobre programación gráfica.

11. BIBLIOGRAFÍA

- **Kenji Kawaida Villegas.** (s. f.). *Ampliación de una ciudad interactiva con OpenGL: actividad práctica de programación gráfica*. Enunciado y código base proporcionados por el docente.
- **The Khronos Group.** (s. f.). *OpenGL Registry*. Especificaciones de OpenGL y GLSL. https://registry.khronos.org/OpenGL/index_gl.php
- **LWJGL.** (s. f.). *Getting Started*. Guía de uso de Lightweight Java Game Library. https://www.lwjgl.org/guide
- **GLFW.** (s. f.). *Documentation*. Documentación sobre ventanas, contextos de OpenGL y entradas del usuario. https://www.glfw.org/documentation.html
- **Apache Software Foundation.** (s. f.). *Maven Getting Started Guide*. https://maven.apache.org/guides/getting-started/
- **Oracle.** (s. f.). *Java Platform, Standard Edition 17: API Specification*. Referencia de Java, incluyendo gráficos y sonido. https://docs.oracle.com/en/java/javase/17/docs/api/